PROJECT REPORT

Course Title: Deep Learning

Project Title: Pakistan Sign Language (PSL) Recognition System

Submitted By:

- **Name:** Muhammad Rayyan Tariq
- **Roll Number:** SU92-BSCSM-F23-237
- **Class / Section:** BS Computer Science (BSCS - Section 7A)
- **Institution:** Superior University, Lahore
- **Date:** October 2026

1. INTRODUCTION & BACKGROUND

Communication accessibility remains one of the most persistent societal challenges for the hearing-impaired community. In Pakistan, hundreds of thousands of individuals rely on Pakistan Sign Language (PSL) as their primary visual-gestural mode of communication. However, the scarcity of certified sign language interpreters across public institutions, educational facilities, healthcare centers, and customer service sectors severely restricts independent civic participation for hearing-impaired citizens.

While computer vision and deep learning have catalyzed massive breakthroughs in automated sign language translation globally, the overwhelming majority of existing academic literature and open-source models focus exclusively on American Sign Language (ASL) or British Sign Language (BSL). Pakistan Sign Language possesses distinct regional hand shapes, cultural nuances, and alphabet structures that cannot be accurately parsed by foreign models.

To bridge this technological gap, this project implements an end-to-end deep learning system designed to automate the recognition of **37 Pakistan Sign Language alphabet and letter classes**. By comparing a custom-engineered Convolutional Neural Network (CNN) built from scratch against a state-of-the-art Transfer Learning model (MobileNetV2), this study evaluates trade-offs in parameter efficiency, convergence speed, and generalization capability.

2. PROBLEM STATEMENT

Manual interpretation of sign language is inherently limited by human availability, cost, and geographical barriers. Developing an automated visual recognition system for Pakistan Sign Language presents several distinct technical hurdles:

1. **Intra-Class Variance and Inter-Class Similarity:** Different sign gestures in PSL can exhibit subtle geometric variations in finger placement and orientation, making feature extraction highly sensitive to network depth.
2. **Environmental Noise:** Real-world deployment requires robustness against varying ambient lighting conditions, skin tones, and background clutter.
3. **Absence of Standardized Digital Repositories:** Unlike ASL, PSL lacks massive commercial datasets, requiring robust processing of available benchmark collections to prevent overfitting.
4. **Architectural Selection:** Determining whether a lightweight custom CNN trained from scratch can compete with pre-trained ImageNet feature extractors under constrained computational budgets.

3. DATASET SPECIFICATION & PIPELINE OPTIMIZATION

3.1 Dataset Origin & Composition

The system was trained and evaluated using Wasif Ullah's benchmark Kaggle dataset titled *"Dataset Pakistani Sign Language"*. The dataset comprises a total of **7,242 RGB images** distributed uniformly across **37 distinct classes** representing the core PSL alphabet and letter set (e.g., *Aliph, Bay, Tey*, etc.).

3.2 Data Splitting Strategy

To ensure rigorous evaluation without data leakage, the dataset was partitioned using an **80/20 stratified split**:

- **Training Partition:** 5,794 images utilized for weight updates via backpropagation.
- **Validation Partition:** 1,448 images held out exclusively for unbiased performance testing during epochs.

3.3 Tensor Preprocessing & Performance Pipeline

Raw images were standardized to uniform dimensions of 128 x 128 x 3 pixels to balance spatial resolution retention with matrix multiplication efficiency. To eliminate CPU-GPU I/O bottlenecks during execution, TensorFlow's `tf.data` API was deployed with advanced pipeline optimizations:

- **Disk Caching (`cache()`):** Stores decoded images in memory after the first epoch, preventing redundant disk reads.
- **Dynamic Prefetching (`prefetch(AUTOTUNE)`):** Overlaps data preprocessing and model execution steps asynchronously, maximizing hardware accelerator utilization.

4. SYSTEM ARCHITECTURE & NETWORK DESIGN

To establish a comprehensive comparative baseline, two distinct architectural paradigms were constructed.

4.1 Model A: Custom Convolutional Neural Network (CNN Built from Scratch)

Model A was designed entirely from scratch to evaluate how effectively a network can learn low-level spatial hierarchies (edges, corners, textures) and high-level semantic shapes without pre-trained priors.

- **Input Normalization:** A `Rescaling(1./255)` layer maps raw pixel integers from [0, 255] to floating-point values in [0.0, 1.0].
- **Feature Extraction Blocks:** Comprises three sequential convolutional blocks:
    - *Block 1:* 32 filters (3 x3 kernel, padding='same') >> Rectified Linear Unit (ReLU) activation >> MaxPooling2D (2 x 2 pool size) for spatial dimensionality reduction.
    - *Block 2:* 64 filters (3 x 3 kernel, padding='same') >> ReLU activation >> MaxPooling2D (2 x 2).
    - *Block 3:* 128 filters (3 x 3 kernel, padding='same') >> ReLU activation >> MaxPooling2D (2 x 2).
- **Classification Head:** A Flatten layer transforms 3D feature maps into a 1D vector >> Fully Connected Dense layer (256 units, ReLU activation) >> Dropout regularization (rate = 0.5) to randomly deactivate units and prevent co-adaptation >> Output Dense layer (37 units, Softmax activation) producing a probability distribution across all sign classes.

- **Compilation Parameters:** Optimized using the **Adam optimizer** (learning rate = 0.001) and **Sparse Categorical Crossentropy** loss.

4.2 Model B: Transfer Learning using MobileNetV2

Model B leverages an established pre-trained architecture to examine transfer learning efficiency and feature reusability.

- **Base Model:** MobileNetV2 instantiated with `include_top=False` and input shape (128, 128, 3), utilizing pre-trained ImageNet weights.
- **Freezing Strategy:** The base feature extraction layers were frozen (`base_model.trainable = False`) to preserve universal edge and texture filters learned from millions of general images, reducing trainable parameters and preventing catastrophic forgetting.
- **Input Normalization:** Scaled via `Rescaling(1./127.5, offset=-1)` mapping input tensors to [-1, 1] to match MobileNetV2 preprocessing requirements.
- **Custom Classification Head:** Global Average Pooling 2D (collapsing spatial dimensions) >> Dropout regularization (rate = 0.3) >> Dense layer (128 units, ReLU activation) >> Output Dense layer (37 units, Softmax activation).
- **Compilation Parameters:** Optimized using the **Adam optimizer** and **Sparse Categorical Crossentropy** loss.

5. IMPLEMENTATION & TRAINING DYNAMICS

The system was implemented in Python using TensorFlow/Keras within an accelerated Google Colab environment powered by an NVIDIA T4 GPU.

5.1 Training Execution

- **Model A (Custom CNN):** Trained for 10 epochs. The network demonstrated stable gradient descent, steadily decreasing training and validation loss while converging to high accuracy.
- **Model B (Transfer Learning):** Trained for 6 epochs. Due to the rich semantic priors inherited from the frozen ImageNet base, the model exhibited immediate convergence and plateaued at near-optimal accuracy within the first few epochs.
- **Model Serialization:** Upon completion, both trained models were saved to persistent Google Drive storage as binary HDF5 weights (`psl_custom_cnn.h5` and `psl_transfer_learning.h5`).

6. EXPERIMENTAL RESULTS & EVALUATION

6.1 Quantitative Performance Summary

The models were evaluated against the validation dataset using accuracy and loss metrics.

Model Architecture	Underlying Paradigm	Training Epochs	Validation Accuracy	Validation Loss	Total Parameters
Model A	Custom CNN (Scratch)	10	99.31%	0.0241	~1.2 Million
Model B	MobileNetV2 (Transfer)	6	**99.93%**	**0.0035**	2,426,725 (Trainable: ~168K)

6.2 Analysis of Evaluation Curves

- **Loss & Accuracy Curves:** Training and validation accuracy curves for Model A showed steady upward progression, crossing 99% by epoch 9 without severe divergence between training and validation metrics, indicating effective regularization via Dropout.
- **Convergence Efficiency:** Model B achieved a validation accuracy exceeding 99.5% by Epoch 2, proving that transferring convolutional feature extractors drastically reduces the required training duration for domain-specific sign language tasks.

7. DISCUSSION

The experimental results provide clear technical insights into computer vision applications for regional sign languages:

1. **Feasibility from Scratch:** Building a custom CNN (Model A) proved entirely viable for closed-vocabulary sign recognition, successfully achieving 99.31% accuracy. This confirms that a modest 3-block convolutional topology is sufficient to learn distinct hand geometry when image preprocessing is standardized.
2. **Superiority of Transfer Learning:** Model B outperformed Model A in both accuracy (99.93% vs. 99.31%) and training speed (6 epochs vs. 10 epochs). By utilizing pre-trained convolutional filters, the network bypassed the initial phase of learning basic edge detection and immediately focused on high-level gesture classification.

8. CONCLUSION & FUTURE WORK

This study successfully developed, trained, and evaluated an end-to-end deep learning system for 37 Pakistan Sign Language classes. The comparative evaluation demonstrates that while custom architectures built from scratch provide reliable performance, transfer learning models like MobileNetV2 deliver superior generalization and rapid convergence.

Future Scope:

- **Continuous Sign Translation:** Expanding the pipeline from static alphabet frames to dynamic video sequences utilizing Recurrent Neural Networks (LSTM/GRU) or 3D CNNs.
- **Edge Deployment:** Converting the trained models to TensorFlow Lite (`.tflite`) for deployment on mobile applications, enabling real-time assistive translation on consumer smartphones.